



FRIDGE MANAGER

Sistema informativo per la gestione intelligente degli alimenti domestici

Documentazione di Progetto

Esame di Basi di Dati - Antonino Valese

Modello informativo, scelte progettuali e applicazione web

Backend: Django • Database: MySQL • Frontend: Bootstrap

Indice

Indice	2
1. Introduzione e contesto applicativo	3
2. Analisi dei requisiti	3
2.1 Requisiti funzionali	3
2.2 Requisiti informativi	3
3. Progettazione concettuale (modello E-R)	5
3.1 Entità individuate	5
3.2 Descrizione delle entità e degli attributi	5
Utente	5
Frigorifero	5
Categoria	5
Prodotto	5
FridgeItem	5
Ricetta	5
IngredienteRicetta	6
Notifica	6
3.3 Relazioni e cardinalità	6
3.4 Diagramma Entità-Relazione	6
3.5 Generalizzazione / specializzazione	7
3.5.1 Lettura come generalizzazione (bottom-up) e relativi vincoli	7
3.5.2 Lettura come specializzazione (top-down) e relativi vincoli	8
4. Progettazione logica (modello relazionale)	9
4.1 Strategie di traduzione della gerarchia	9
Strategia A – Accorpamento delle figlie nel padre (tabella unica)	9
Strategia B – Accorpamento del padre nelle figlie	9
Strategia C – Sostituzione della gerarchia con relazioni	9
4.2 Scelta motivata	10
4.3 Schema relazionale finale	10
4.4 Vincoli rilevanti	10
Vincoli di integrità referenziale	10
Vincoli di dominio e di chiave	10
Vincolo di business sulla gerarchia	11
4.5 Semplificazioni e adattamenti	11
5. Implementazione del sistema informativo	12
5.1 Stack tecnologico	12
5.2 Mappatura sul Django ORM	12
5.3 Funzionalità realizzate	12
5.4 Organizzazione dell'applicazione	13
5.5 Interfaccia utente (UX)	13
5.6 Popolamento del database	13
6. Sicurezza del sistema (sezione bonus)	14
6.1 SQL injection	14

6.2 Attacco a dizionario e brute-force sul login	14
6.3 Altre protezioni offerte dal framework	14
7. Consegna e contenuto del repository	14
7.1 Istruzioni di installazione (sintesi)	15
8. Conclusioni	15

1. Introduzione e contesto applicativo

Il progetto Fridge Manager consiste nella realizzazione di un sistema informativo completo per la gestione intelligente degli alimenti domestici, con l'obiettivo di sostituire le tradizionali procedure manuali (annotazioni su carta, controllo a vista delle scadenze) con un'applicazione web intuitiva e dall'aspetto curato. Il cuore del sistema è un database relazionale, implementato in MySQL, che ospita tutte le informazioni necessarie per gestire i prodotti, i frigoriferi, gli utenti e i servizi offerti.

Il dominio applicativo è quello della gestione domestica degli alimenti. Ogni utente registrato può creare uno o più frigoriferi virtuali e popolarli con i prodotti effettivamente presenti, indicandone quantità e data di scadenza. Per ciascun prodotto il sistema conosce immediatamente il suo stato: disponibile, consumato oppure scaduto. I frigoriferi possono essere condivisi con altri utenti (ad esempio i componenti di una stessa famiglia o i coinquilini), con ruoli differenziati: chi crea il frigorifero ne è il proprietario, mentre gli utenti invitati ne sono membri.

Il sistema gestisce inoltre le notifiche automatiche relative ai prodotti in scadenza, organizza i prodotti per categorie tematiche e offre una sezione dedicata alle ricette, con la possibilità di associare a ciascuna ricetta gli ingredienti necessari e di suggerire le ricette realizzabili con i prodotti disponibili.

L'obiettivo progettuale è partire da un'analisi rigorosa del dominio, costruire un modello concettuale Entità-Relazione completo (comprensivo di una gerarchia di generalizzazione/specializzazione), derivarne un modello logico relazionale normalizzato e ben vincolato, e infine implementare il sistema come applicazione web con Django, template Django e Bootstrap, curando anche l'esperienza d'uso con un'interfaccia in stile applicazione mobile-web.

2. Analisi dei requisiti

2.1 Requisiti funzionali

Il sistema deve consentire le seguenti operazioni:

- registrazione, autenticazione (login/logout) e gestione delle credenziali personali degli utenti;
- creazione di uno o più frigoriferi da parte di un utente;
- condivisione di un frigorifero con altri utenti, con distinzione del ruolo (proprietario / membro);
- inserimento, modifica e rimozione dei prodotti alimentari all'interno di un frigorifero;
- organizzazione dei prodotti tramite categorie tematiche;
- gestione della quantità disponibile e della data di scadenza di ciascun prodotto;
- aggiornamento dello stato del prodotto (disponibile, consumato, scaduto);
- rilevazione automatica dei prodotti scaduti e in scadenza, con generazione delle relative notifiche;
- presentazione di una dashboard riepilogativa con statistiche e avvisi;
- gestione delle ricette e associazione degli ingredienti necessari;
- suggerimento delle ricette realizzabili in base ai prodotti disponibili nel frigorifero.

2.2 Requisiti informativi

Il database deve memorizzare in modo persistente:

- i dati anagrafici e di accesso degli utenti;
- le informazioni sui frigoriferi e sulle relative condivisioni (chi vi accede e con quale ruolo);
- l'anagrafica dei prodotti alimentari e le categorie a cui appartengono;
- gli elementi presenti in ciascun frigorifero, con quantità, data di scadenza, data di inserimento e stato;
- le ricette e gli ingredienti ad esse associati con le relative quantità;
- le notifiche generate dal sistema e il loro stato di lettura.

3. Progettazione concettuale (modello E-R)

A partire dalla descrizione del dominio è stato costruito un modello concettuale Entità-Relazione completo. In questa sezione vengono presentate le entità individuate, gli attributi, le relazioni con le relative cardinalità e, in modo approfondito, la gerarchia di generalizzazione/specializzazione con l'analisi dei vincoli associati.

3.1 Entità individuate

Entità	Descrizione
Utente	Persona registrata che accede al sistema con credenziali personali.
Owner / Member	Sottoclassi di Utente che ne specializzano il ruolo (vedi § 3.5).
Frigorifero	Contenitore logico creato da un utente, condivisibile con altri.
Categoria	Classificazione tematica dei prodotti (es. Bevande, Verdura, Latticini).
Prodotto	Anagrafica di un alimento, indipendente dal frigorifero.
FridgeItem	Istanza di un prodotto presente in uno specifico frigorifero.
Ricetta	Preparazione culinaria con istruzioni e ingredienti.
IngredienteRicetta	Associazione tra una ricetta e un prodotto, con la quantità necessaria.
Notifica	Avviso generato dal sistema e destinato a un utente.

3.2 Descrizione delle entità e degli attributi

Utente

Rappresenta l'utente registrato. Attributi: id (identificatore), username, email (univoca), password (memorizzata in forma cifrata).

Frigorifero

Rappresenta un frigorifero virtuale. Attributi: id, nome, data_creazione. Il legame con gli utenti (proprietario e membri) è espresso tramite la relazione Condivisione (§ 3.4).

Categoria

Classifica i prodotti. Attributi: id, nome. Esempi di istanze: Bevande, Verdura, Carne, Latticini. A livello applicativo a ciascuna categoria è associata un'icona rappresentativa, usata nell'interfaccia del frigo visuale.

Prodotto

Anagrafica di un alimento, indipendente dalla sua presenza fisica in un frigorifero. Attributi: id, nome. È collegato a una sola categoria.

FridgeItem

Rappresenta la presenza concreta di un prodotto in un determinato frigorifero. Attributi: id, quantità, data_scadenza, data_inserimento, stato. Lo stato può assumere i valori: disponibile, consumato, scaduto.

Ricetta

Preparazione culinaria. Attributi: id, nome, istruzioni.

IngredienteRicetta

Entità associativa che lega una ricetta a un prodotto. Attributi: id, quantita_necessaria.

Notifica

Avviso destinato a un utente, tipicamente generato a partire da un Fridgeltem in scadenza. Attributi: id, messaggio, data_creazione, letta (booleano).

3.3 Relazioni e cardinalità

Relazione	Entità coinvolte	Cardinalità	Significato
Condivisione	Utente – Frigorifero	N:N	Un utente accede a più frigoriferi; un frigorifero è condiviso tra più utenti. Porta l'attributo ruolo.
Contiene	Frigorifero – Fridgeltem	1:N	Un frigorifero contiene molti elementi; ogni elemento appartiene a un solo frigorifero.
Classifica	Categoria – Prodotto	1:N	Una categoria classifica molti prodotti; un prodotto ha una sola categoria.
Riferisce	Prodotto – Fridgeltem	1:N	Un prodotto compare in molti elementi; ogni elemento riferisce un solo prodotto.
Ingrediente	Ricetta – Prodotto	N:N	Una ricetta usa più prodotti; un prodotto compare in più ricette. Porta quantita_necessaria.
Genera	Fridgeltem – Notifica	1:N	Un elemento può generare più notifiche; ogni notifica deriva da un elemento.
Riceve	Utente – Notifica	1:N	Un utente riceve più notifiche; ogni notifica è destinata a un solo utente.

3.4 Diagramma Entità-Relazione

Il diagramma seguente sintetizza il modello concettuale. I rettangoli rappresentano le entità, i rombi le relazioni (con eventuali attributi), il nodo ISA la gerarchia di generalizzazione/specializzazione. Le cardinalità sono espresse nella notazione (min, max).

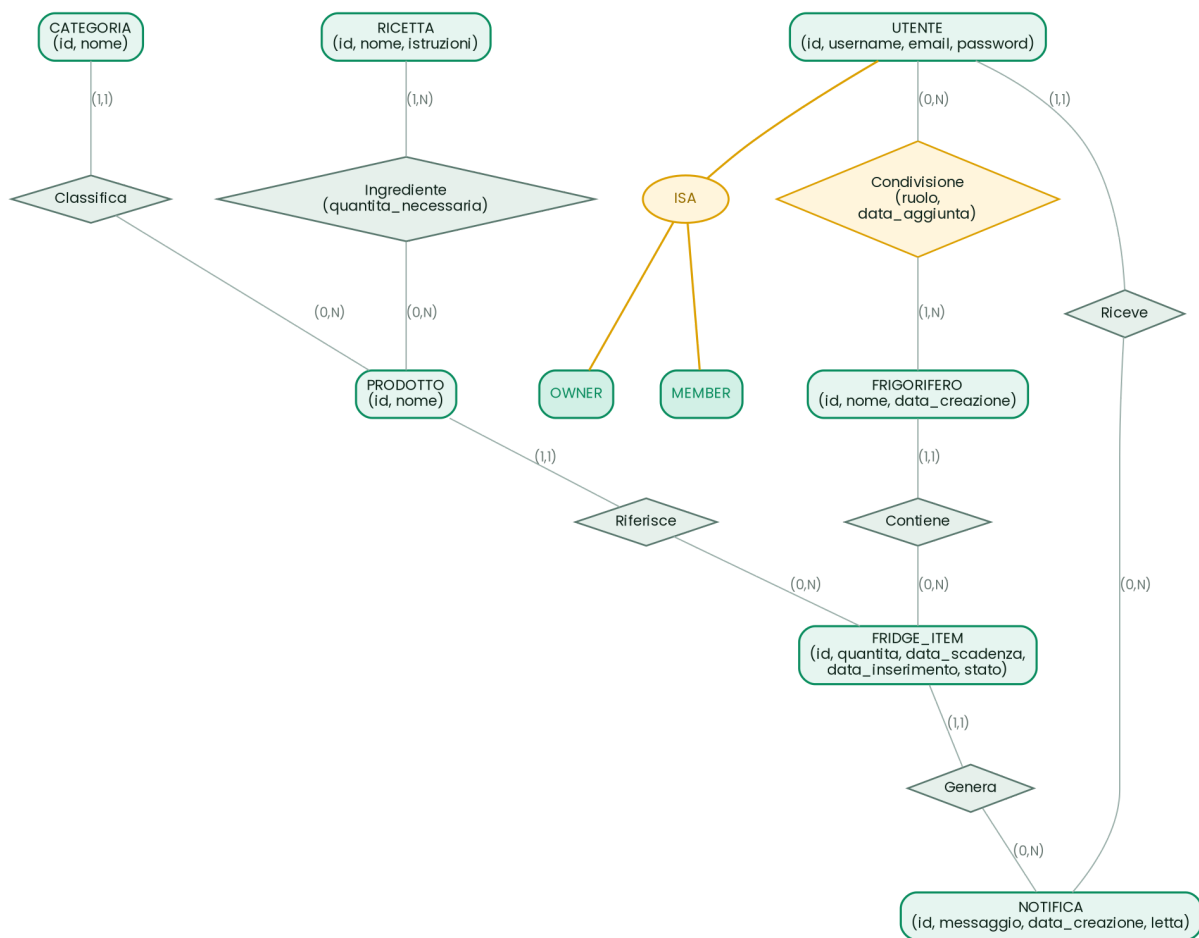


Figura 1 – Schema concettuale E-R del sistema Fridge Manager

3.5 Generalizzazione / specializzazione

Nel dominio sono presenti due tipologie di utente che si comportano in modo differente rispetto a un frigorifero: il proprietario, che lo ha creato e ne gestisce i permessi, e il membro, che vi accede su invito. Per rappresentare questa distinzione è stata introdotta nel modello concettuale una gerarchia di generalizzazione/specializzazione che ha Utente come superclasse (padre) e Owner e Member come sottoclassi (figlie).

La superclasse **Utente** raccoglie gli attributi comuni a tutti gli utenti (id, username, email, password). Le sottoclassi specializzano il comportamento:

- **Owner** – può creare frigoriferi, invitare nuovi membri e gestire i permessi di accesso;
- **Member** – può consultare i prodotti di un frigorifero condiviso, aggiornarne le quantità e visualizzare le scadenze.

3.5.1 Lettura come generalizzazione (bottom-up) e relativi vincoli

Letta in chiave di generalizzazione (approccio bottom-up), la gerarchia nasce osservando che Owner e Member condividono numerose proprietà (le credenziali, i dati anagrafici, la capacità di ricevere notifiche). Tali proprietà comuni vengono fattorizzate verso l'alto in un'unica entità padre Utente, evitando duplicazioni. Il vincolo fondamentale della generalizzazione è l'ereditarietà: ogni proprietà e ogni relazione definita sul padre (ad esempio la relazione Riceve con Notifica) vale automaticamente

per tutte le istanze delle figlie. Un'occorrenza di Owner o di Member è, a tutti gli effetti, anche un'occorrenza di Utente.

3.5.2 Lettura come specializzazione (top-down) e relativi vincoli

Letta in chiave di specializzazione (approccio top-down), si parte dall'entità generale Utente e se ne distinguono i sottotipi in base al ruolo. Una gerarchia di specializzazione è caratterizzata da due vincoli ortogonali, che ne definiscono la copertura:

- **vincolo di completezza (totale / parziale):** indica se ogni istanza del padre deve appartenere ad almeno una figlia (totale) oppure se può non appartenere ad alcuna (parziale);
- **vincolo di disgiunzione (esclusiva / sovrapposta):** indica se un'istanza del padre può appartenere ad al massimo una figlia (esclusiva o disgiunta) oppure a più figlie contemporaneamente (sovrapposta).

Nel caso del Fridge Manager l'analisi del dominio porta a una conclusione precisa e importante: la specializzazione è parziale e sovrapposta.

- **È parziale** perché un utente appena registrato che non ha ancora creato né ricevuto in condivisione alcun frigorifero non è né Owner né Member: esiste come Utente senza appartenere ad alcuna sottoclasse;
- **È sovrapposta** perché lo stesso utente può essere contemporaneamente proprietario di un frigorifero (Owner) e membro di un altro frigorifero condiviso da un amico (Member).

Questa osservazione è il punto cardine dell'intera progettazione: poiché il ruolo non è una proprietà assoluta dell'utente, ma dipende dallo specifico frigorifero a cui si riferisce, il ruolo appartiene concettualmente alla relazione tra Utente e Frigorifero, non all'entità Utente. Questa considerazione guida la scelta di traduzione adottata in fase di progettazione logica (§ 4).

4. Progettazione logica (modello relazionale)

In questa fase il modello E-R viene tradotto in uno schema logico relazionale. Il punto più delicato è la traduzione della gerarchia Utente → Owner/Member. Vengono quindi presentate le strategie classiche di ristrutturazione, con i relativi vincoli e i pro e contro, per poi motivare la scelta finale.

4.1 Strategie di traduzione della gerarchia

La teoria delle basi di dati prevede tre tecniche standard per eliminare una gerarchia di generalizzazione in fase di progettazione logica.

Strategia A – Accorpamento delle figlie nel padre (tabella unica)

Tutte le entità della gerarchia confluiscono in un'unica tabella, con un attributo discriminatore tipo che indica il ruolo. È l'equivalente della Single Table Inheritance.

Vincolo associato: un vincolo di dominio CHECK (tipo IN ('owner','member')) garantisce la coerenza del discriminatore; gli attributi specifici di una sola figlia possono assumere valore NULL per le istanze dell'altra.

- **Vantaggi:** struttura semplice, nessun JOIN, query più veloci.
- **Svantaggi:** presenza di valori NULL, minore normalizzazione, difficoltà a esprimere il caso sovrapposto (un solo valore di tipo per riga non basta se un utente è sia owner sia member).

Strategia B – Accorpamento del padre nelle figlie

Si eliminano la superclasse e si replicano i suoi attributi in ciascuna tabella figlia (Owners e Members).

Vincolo associato: è applicabile solo se la copertura è totale, altrimenti le istanze del padre che non appartengono ad alcuna figlia non troverebbero collocazione.

- **Vantaggi:** nessuna tabella padre da interrogare per i dati specifici.
- **Svantaggi:** duplicazione degli attributi comuni; nel caso sovrapposto i dati di un utente che è sia owner sia member verrebbero ripetuti in due tabelle, con rischio di incoerenza. Non adatto a copertura parziale.

Strategia C – Sostituzione della gerarchia con relazioni

Si mantiene la tabella padre Utente e si rappresentano le figlie tramite relazioni. Nel nostro caso, poiché è emerso che il ruolo dipende dal frigorifero, la sostituzione assume la forma più naturale: una relazione associativa Condivisione tra Utente e Frigorifero che porta l'attributo ruolo. Questo è l'equivalente concreto e ottimizzato della tecnica di sostituzione.

Vincoli associati: integrità referenziale verso Utente e Frigorifero; vincolo di dominio CHECK (ruolo IN ('owner','member')); chiave primaria composta (utente_id, frigorifero_id) per evitare condivisioni duplicate.

- **Vantaggi:** gestisce nativamente sia la copertura parziale (l'utente può non comparire in alcuna condivisione) sia quella sovrapposta (lo stesso utente può avere ruolo owner su un frigorifero e member su un altro); massima normalizzazione; implementa direttamente la funzionalità di condivisione.
- **Svantaggi:** introduce un JOIN in più per ricostruire il ruolo, costo del tutto trascurabile.

4.2 Scelta motivata

Viene adottata la Strategia C (sostituzione della gerarchia con la relazione Condivisione). La motivazione è diretta: l'analisi concettuale (§ 3.5.2) ha dimostrato che la specializzazione è parziale e sovrapposta e che il ruolo è una proprietà della relazione utente–frigorifero, non dell'utente. Le strategie A e B sono pensate per gerarchie disgiunte e mal si adattano a un utente che è simultaneamente owner e member. La Strategia C, invece, modella la realtà in modo fedele, mantiene lo schema normalizzato, evita valori NULL e duplicazioni e, come effetto positivo, realizza direttamente il requisito di condivisione dei frigoriferi. È inoltre la soluzione che si mappa più naturalmente sull'ORM di Django tramite un modello associativo (through model).

4.3 Schema relazionale finale

Notazione: PK = chiave primaria, FK = chiave esterna, U = univoco.

```
Utente(id PK, username, email U, password)

Frigorifero(id PK, nome, data_creazione)

Condivisione(utente_id FK -> Utente.id,
              frigorifero_id FK -> Frigorifero.id,
              ruolo, data_aggiunta)
  PK = (utente_id, frigorifero_id)

Categoria(id PK, nome)

Prodotto(id PK, nome, categoria_id FK -> Categoria.id)

FridgeItem(id PK,
            frigorifero_id FK -> Frigorifero.id,
            prodotto_id FK -> Prodotto.id,
            quantita, data_scadenza, data_inserimento, stato)

Ricetta(id PK, nome, istruzioni)

IngredienteRicetta(id PK,
                   ricetta_id FK -> Ricetta.id,
                   prodotto_id FK -> Prodotto.id,
                   quantita_necessaria)

Notifica(id PK,
          utente_id FK -> Utente.id,
          fridge_item_id FK -> FridgeItem.id,
          messaggio, data_creazione, letta)
```

4.4 Vincoli rilevanti

Vincoli di integrità referenziale

Ogni chiave esterna deve riferire una riga esistente nella tabella di destinazione:

- categoria_id in Prodotto deve esistere in Categoria;
- frigorifero_id e prodotto_id in FridgeItem devono esistere rispettivamente in Frigorifero e Prodotto;
- utente_id e frigorifero_id in Condivisione devono esistere nelle rispettive tabelle;
- utente_id e fridge_item_id in Notifica devono esistere in Utente e FridgeItem.

Vincoli di dominio e di chiave

- email in Utente è univoca (vincolo UNIQUE);
- quantita in Fridgeltem e quantita_necessaria in IngredienteRicetta devono essere maggiori di zero;
- data_scadenza, alla data di inserimento, deve essere successiva alla data corrente;
- stato in Fridgeltem è limitato all'insieme {disponibile, consumato, scaduto};
- ruolo in Condivisione è limitato all'insieme {owner, member};
- la coppia (utente_id, frigorifero_id) in Condivisione è chiave primaria: un utente non può essere associato due volte allo stesso frigorifero.

Vincolo di business sulla gerarchia

Per ogni frigorifero deve esistere esattamente una condivisione con ruolo owner (il creatore). Si tratta di un vincolo applicativo, gestito a livello di logica Django al momento della creazione del frigorifero, poiché non esprimibile con una semplice CHECK di colonna.

4.5 Semplificazioni e adattamenti

In coerenza con la richiesta di mantenere il sistema essenziale, sono state adottate alcune semplificazioni motivate:

- le entità associative Condivisione e IngredienteRicetta, derivate da relazioni N:N, sono state tradotte in tabelle dedicate con chiave propria o composta, come previsto dalla teoria relazionale;
- la gerarchia Owner/Member non genera tabelle separate ma viene riassorbita nell'attributo ruolo di Condivisione, come motivato in § 4.2;
- Prodotto rappresenta l'anagrafica astratta dell'alimento, mentre Fridgeltem ne rappresenta l'istanza fisica in un frigorifero: questa separazione evita la duplicazione dei dati anagrafici e consente di riutilizzare lo stesso prodotto in più frigoriferi e in più ricette;
- la gestione della password è delegata al framework, che ne memorizza esclusivamente l'hash (vedi § 7).

5. Implementazione del sistema informativo

5.1 Stack tecnologico

Tecnologia	Ruolo
Python	Linguaggio di programmazione principale.
Django	Framework backend; gestione di modelli, viste, URL e autenticazione.
Django ORM	Mappatura oggetto-relazionale tra le classi Python e le tabelle MySQL.
Template Django	Generazione lato server delle pagine HTML, con template tag personalizzati.
MySQL	Sistema di gestione del database relazionale.
Bootstrap 5	Libreria CSS per l'interfaccia grafica responsive.
JavaScript	Utilizzato solo dove indispensabile (bundle di Bootstrap, conferme di eliminazione).

5.2 Mappatura sul Django ORM

Lo schema relazionale si traduce direttamente in modelli Django. La gerarchia risolta tramite relazione diventa un modello associativo (through) sulla relazione multi-a-multi tra Utente e Frigorifero:

- ogni entità → una classe che eredita da `models.Model`;
- le chiavi esterne → campi `ForeignKey` con `on_delete` appropriato;
- la relazione `Condivisione` → `ManyToManyField(Utente, through='Condivisione')` con il campo `ruolo` come scelta limitata (`choices`);
- i vincoli di dominio → `choices`, `validators` (es. `MinValueValidator` per le quantità), attributo `unique` e `UniqueConstraint`;
- l'autenticazione → si appoggia al modello `User` integrato di Django, con cui coincide l'entità `Utente`.

Lo stato di scadenza degli elementi non è memorizzato come dato ridondante ma calcolato a runtime tramite proprietà del modello (`giorni alla scadenza`, `in scadenza`, `scaduto`), garantendo che l'informazione sia sempre coerente con la data corrente.

5.3 Funzionalità realizzate

La traccia richiede l'implementazione di almeno quattro funzionalità. L'applicazione ne realizza cinque principali, che coprono il ciclo di vita completo dei dati modellati:

1. Registrazione e autenticazione degli utenti (`login`, `logout`, `registrazione`).
2. Creazione e condivisione dei frigoriferi, con assegnazione del ruolo proprietario/membro.
3. Gestione dei prodotti in frigorifero (`inserimento`, `aggiornamento quantità/stato`, `rimozione`) con visualizzazione per categoria.
4. Controllo automatico delle scadenze e generazione delle relative notifiche.
5. Gestione delle ricette e suggerimento delle ricette realizzabili in base agli ingredienti disponibili.

5.4 Organizzazione dell'applicazione

Il progetto segue l'architettura MVT (Model-View-Template) di Django:

- **Model:** le classi che rappresentano le tabelle del database (models.py);
- **View:** la logica applicativa che elabora le richieste e interroga i modelli (views.py);
- **Template:** i file HTML con i tag di template Django, i template tag personalizzati e le classi Bootstrap per la presentazione;
- **URLconf:** la mappatura tra gli indirizzi e le viste (urls.py).

5.5 Interfaccia utente (UX)

L'applicazione è stata curata anche dal punto di vista dell'esperienza d'uso, con un'estetica coerente in stile applicazione mobile-web costruita interamente con Bootstrap e CSS personalizzato (palette verde menta, card arrotondate, logo dedicato). Gli elementi principali dell'interfaccia sono:

- **Dashboard riepilogativa:** la home presenta un saluto personalizzato, quattro indicatori sintetici (frigoriferi, prodotti disponibili, prodotti in scadenza, prodotti scaduti) e un'anteprima visuale dei prodotti più critici.
- **Frigo visuale:** all'interno di un frigorifero i prodotti sono mostrati come schede (card) raggruppate per categoria, ciascuna con una striscia colorata che ne comunica a colpo d'occhio lo stato (disponibile, in scadenza, scaduto, consumato), la quantità e la data di scadenza.
- **Avvisi di scadenza automatici:** banner generati dinamicamente a ogni visita segnalano i prodotti scaduti o in scadenza, senza necessità di processi pianificati esterni.
- **Login curato:** schermata di accesso centrata e minimale, con il logo in evidenza e i campi corredati di icone.
- **Navigazione mobile:** su dispositivi piccoli compare una barra di navigazione inferiore in stile app, con l'indicatore del numero di avvisi non letti.

5.6 Popolamento del database

Per consentire l'avvio immediato e la dimostrazione del sistema sono forniti dati di esempio realistici. Sono disponibili due modalità equivalenti:

- **comando di gestione popola_db:** carica utenti, frigoriferi condivisi, un catalogo di circa trenta prodotti, ricette ed elementi con scadenze calcolate rispetto alla data corrente (alcuni già scaduti, alcuni in scadenza, molti validi), generando anche le notifiche; l'opzione --reset svuota e ricarica i dati;
- **fixture dati_esempio.json:** un file importabile con il comando loaddata, contenente lo stesso insieme di dati (utenti con password già cifrata, frigoriferi, prodotti, ricette, elementi e notifiche).

Gli utenti di esempio (mario, giulia, luca, admin) condividono la password Password123! e permettono di provare immediatamente i diversi ruoli e le funzionalità di condivisione.

6. Sicurezza del sistema (sezione bonus)

La traccia propone, come attività facoltativa, di mostrare come alcune vulnerabilità comuni possano essere sfruttate e, soprattutto, come prevenirle. Questa sezione ha finalità esclusivamente didattica e difensiva: tutte le prove vanno eseguite in ambiente locale e controllato, sul proprio progetto. L'accento è posto sulle contromisure.

6.1 SQL injection

La SQL injection si verifica quando l'input dell'utente viene concatenato direttamente in una query SQL, permettendo di alterarne la logica. Il rischio nasce quando si costruiscono query come stringhe a mano.

Prevenzione: il Django ORM costruisce query parametrizzate, in cui i valori forniti dall'utente sono trattati come parametri e non come codice SQL eseguibile. Utilizzando esclusivamente i metodi dell'ORM (filter, get, create) anziché query SQL grezze, l'input viene automaticamente neutralizzato. La dimostrazione consiste nel mostrare che una stringa malevola inserita in un campo di ricerca viene salvata o ignorata come testo, senza alcun effetto sulla struttura della query.

6.2 Attacco a dizionario e brute-force sul login

L'attacco a dizionario prova un elenco di password comuni; l'attacco brute-force prova in modo esaustivo molte combinazioni. Entrambi mirano a indovinare le credenziali di accesso.

Prevenzione: Django non memorizza mai le password in chiaro, ma soltanto il loro hash calcolato con un algoritmo robusto e con salt (per impostazione predefinita PBKDF2), il che rende inefficace il furto del database. Per contrastare i tentativi ripetuti sul login l'applicazione adotta un meccanismo di limitazione (throttling) che blocca temporaneamente l'accesso dopo alcuni tentativi falliti, affiancato dai validatori di robustezza delle password di Django. La dimostrazione consiste nel mostrare che, dopo un numero prefissato di tentativi errati, l'accesso da quell'origine viene temporaneamente bloccato.

6.3 Altre protezioni offerte dal framework

- protezione CSRF integrata sui form tramite token;
- escaping automatico dell'output nei template, che mitiga gli attacchi XSS;
- gestione delle sessioni con cookie sicuri;
- controllo degli accessi alle viste tramite decoratori di autenticazione e verifica del ruolo, che impedisce a un utente di accedere a frigoriferi non propri.

7. Consegna e contenuto del repository

Il progetto verrà pubblicato su un repository GitHub pubblico, il cui link sarà condiviso con il docente almeno 4 giorni prima dell'esame. Il repository conterrà:

- la presente documentazione con il modello informativo e le scelte progettuali;
- il codice sorgente completo e funzionante dell'applicazione Django;

- i dati di esempio (comando `popola_db` e fixture `dati_esempio.json`) sufficienti a mostrare il funzionamento del sistema;
- le istruzioni per l'installazione e l'avvio del progetto (file `README`).

7.1 Istruzioni di installazione (sintesi)

```
# 1. Clonare il repository
git clone <url-del-repository>
cd fridge-manager

# 2. Ambiente virtuale e dipendenze
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Configurare il database MySQL in settings.py (non necessario in fase di sviluppo
poiché usiamo sql lite)

# 4. Creare le tabelle
python manage.py makemigrations
python manage.py migrate

# 5a. Popolare con dati realistici (consigliato ma non obbligatorio)
python manage.py popula_db
#      in alternativa, importare la fixture:
# python manage.py loaddata dati_esempio2.json

# 6. Avviare il server
python manage.py runserver
```

Login di esempio: mario / Password123! (oppure giulia, luca, admin).

8. Conclusioni

Il progetto Fridge Manager parte da un'analisi del dominio rigorosa per arrivare a un sistema informativo coerente in ogni suo livello. Il modello concettuale introduce una gerarchia di generalizzazione/specializzazione di cui sono stati analizzati esplicitamente i vincoli di completezza e disgiunzione, riconoscendone la natura parziale e sovrapposta. Questa analisi ha guidato la progettazione logica verso la traduzione più corretta della gerarchia – la sostituzione con la relazione Condivisione – che produce uno schema relazionale normalizzato, ben vincolato e direttamente mappabile sull'ORM di Django. L'implementazione realizza più delle quattro funzionalità richieste utilizzando esclusivamente lo stack indicato (Django, template Django, Bootstrap), arricchita da un'interfaccia curata in stile applicazione, da un database popolato con dati realistici e da una sezione facoltativa che documenta le principali misure di sicurezza adottate.